

The Relationship of Zero, One, and Infinity in Information Processing

The Intrinsic Cardinalities of Computation

Registry: [\[HOWL-INFO-11-2026\]](#)

Series Path: [\[HOWL-INFO-1-2026\]](#) → [\[HOWL-INFO-2-2026\]](#) → [\[HOWL-INFO-3-2026\]](#) → [\[HOWL-INFO-4-2026\]](#) → [\[HOWL-INFO-5-2026\]](#) → [\[HOWL-INFO-6-2026\]](#) → [\[HOWL-INFO-7-2026\]](#) → [\[HOWL-INFO-8-2026\]](#) → [\[HOWL-INFO-10-2026\]](#) → [\[HOWL-INFO-11-2026\]](#)

Date: June 2026

DOI: 10.5281/zenodo.20616701

Domain: Information Theory / Systems Architecture / Foundations of Computation

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

1. The Three Cardinalities

Closed Loop Architecture, described in [\[HOWL-COMP-12-2026\]](#), specifies systems using four flat lists. The first list assigns every noun in the system a cardinality: Zero, One, or Infinity. Zero means the system references the concept but has no runtime representation of it — the concept exists in the specification so that the specification is complete, but no code creates, updates, or destroys instances. One means exactly one instance exists for the lifetime of the system — a singleton. Infinity means a population of instances exists, with members created and destroyed dynamically.

In that paper, cardinality was presented as an architectural tool. Assigning cardinalities to entity groups determined their interaction patterns mechanically: One interacting with Infinity produced fan-out or convergence, Infinity interacting with Infinity produced list-to-list filtering, Zero interacting with anything produced pure event sequencing. The tool was applied to specify a complete operating system, and the cardinality assignments revealed structural properties of OS design that were not previously visible: fifteen singletons orchestrating twenty populations, nine autonomous decision points out of thirty-seven entity groups, failure handling structurally identical to normal operation.

This paper makes a different claim. The three cardinalities are not an organizational tool that a specification method happens to use. They are intrinsic properties of information processing. Any system that processes information — whether implemented in silicon, described in mathematics, executed by biological cognition, or organized in human institutions — must contend with three distinct modes of existence: things it references but cannot operate on, the single unit of actual work, and multiplicities that must be reduced to a single unit before work can proceed. These three modes have fixed natures. Their

interaction patterns are consequences of those natures, not design decisions. Ignoring them does not eliminate them. It produces the specific classes of problems each domain considers hardest.

The paper proceeds by examining each cardinality in isolation, then their interactions, then the consequences of violating their natures, and finally their appearance across domains outside software engineering.

2. One Is the Unit of Work

A central processing unit executes one instruction per cycle per core. The instruction reads from a small number of registers, performs one operation, and writes to a small number of registers. Even single-instruction-multiple-data (SIMD) extensions do not violate this: one instruction is issued, operating on one packed vector treated as a single operand. The vector contains multiple values, but the CPU addresses it as one thing. Multicore processors achieve parallelism by replicating the single-instruction execution unit — the mechanism is N instances of One, not one instance of N.

A human being processes one conscious thought at a time. Cognitive psychology's extensive literature on attention, working memory, and task switching establishes that what appears to be parallel thought is rapid sequential alternation between focal points. A person considering a list of options evaluates them one at a time, or groups them into categories and evaluates each category as one thing, or abstracts the entire list into a summary and evaluates the summary as one thing. At no point does cognition operate on the list as an undifferentiated multiplicity. It reduces the multiplicity to a unit and operates on the unit.

Natural language constructs meaning through one grammatical subject performing one verb on one object at a time. "The scheduler manages processes" contains one subject, one verb, one object. The object "processes" is plural, but it functions as a single grammatical unit — one noun phrase referring to a collective. When precision demands addressing individual members of the collective, language switches to singular reference: "The scheduler selects the highest-priority process." The sentence cannot address all processes simultaneously except by treating them as one abstracted thing.

Mathematical notation exhibits the same property. The summation symbol Σ does not add N numbers simultaneously. It specifies an iteration: for i from 1 to N, accumulate one term at a time. The integral symbol $\int$ specifies a limit of such accumulations. Matrix multiplication specifies a pattern of one-multiply-one-add steps. Every mathematical operation that appears to address a collection actually specifies a procedure for reducing the collection to a unit through iterated single operations.

These observations converge on a single point. One is not a cardinality among three peers. It is the only cardinality at which work occurs. Zero and Infinity are defined in relation to One — Zero is what cannot become One, and Infinity is what must become One before it can be operated on. The three cardinalities are not symmetric. One is the center. The other two describe modes of relationship to that center.

3. Infinity Is Repetition of One

If One is the unit of work, then a population of N entities is N potential units of work. The population has no computational nature independent of its members. It cannot be operated on “as a population” without first being reduced — through iteration, aggregation, filtering, or abstraction — to something that is One.

Consider a concrete example from operating system design. The OS scheduler manages a population of processes. At each scheduling decision, the scheduler does not operate on all processes simultaneously. It examines a run queue — a data structure that is itself one thing — and selects one process to assign to the CPU. The selection involves comparing priority values, checking time slice exhaustion, evaluating CPU affinity. Each comparison involves one process’s data compared against one threshold or one other process’s data. The population is present only as a source from which individual members are drawn for evaluation.

This pattern repeats across every domain examined in [\[HOWL-COMP-12-2026\]](#). In TCP, the stack manages a population of segments awaiting acknowledgment. Retransmission evaluation happens per segment: has this segment’s acknowledgment timer expired? One segment, one timer comparison, one decision. In an internal combustion engine, the ECU manages a population of cylinders. Fuel injection timing is calculated per cylinder: what is this cylinder’s crank angle? One cylinder, one angle, one injection pulse. In a large language model, the forward pass processes a population of attention heads. Each head computes its own key, query, and value projections independently. One head, one matrix multiplication, one attention score.

In every case, a member of the Infinity population becomes One for the duration of the operation performed on it, then returns to undifferentiated membership. The process being scheduled is the focus of the scheduler’s attention for the duration of the context switch evaluation. The segment being checked for retransmission is the focus of the TCP stack’s evaluation for the duration of the timeout check. The cylinder being fired is the focus of the ECU’s calculation for the duration of the injection timing computation.

This temporary promotion to One is not a metaphor. It has concrete manifestations in every computational substrate. In a CPU, promoting a population member to One means loading its data into registers — making it the operand of the current instruction. In human cognition, it means focusing attention on one item — making it the subject of conscious thought. In mathematical notation, it means binding a variable — substituting a specific value for the iteration index.

The passivity of Infinity populations, observed in every CLA specification across radically different domains, is not a design pattern that careful architects happened to follow. It is a consequence of what Infinity is. A population member cannot orchestrate other members because orchestration requires population-level visibility — knowing what all members are doing, comparing across members, making decisions that affect the group. Population-level visibility is the defining characteristic of One. An Infinity member that

acquires population-level visibility has become One, whether or not the system's specification acknowledges this.

The flow patterns that CLA assigns to Infinity interactions are all descriptions of how Infinity becomes One for the purpose of actual computation. Fan-out is iteration: for each member matching a constraint, promote that member to temporary One and apply one operation. Convergence is reduction: each member contributes one partial result, the partial results accumulate into one aggregate, the aggregate is what the permanent One operates on. List-to-list interaction between two Infinity populations is nested iteration: for each in A, for each in B, perform one operation on one pair.

Infinity, as a computational concept, is One with a loop around it.

4. Permanent One and Temporary One

The observation that Infinity members become One during operations, combined with the observation that some entities are One for the system's entire lifetime, produces a distinction that CLA's current cardinality model does not explicitly capture but that is load-bearing in every system.

A permanent One exists from system initialization to system shutdown. The OS kernel, the TCP stack, the car's ECU, the LLM's attention mechanism — these are singletons whose One-ness is structural. They are One because the architecture declares exactly one of them, always present, always available as an orchestration point. Permanent Ones have population-level visibility. They can read from any Infinity population they manage. They can make decisions that affect which population members are promoted to temporary One and what operations are performed on them. They are the source of the iteration that makes Infinity computable.

A temporary One is an Infinity member that has been promoted for the duration of an operation. Process 4,721, currently being evaluated for context switch. TCP segment 88,203, currently being checked for retransmission timeout. Cylinder 3, currently receiving its fuel injection pulse. The promotion is real — the member's data is loaded, its state is read and potentially modified, it is the subject of computation. The promotion is also bounded — when the operation completes, the member returns to the population, and the next member is promoted.

The entire execution of any system is a permanent One generating a stream of temporary Ones from Infinity populations. This is what the execution loop looks like at the most fundamental level: the permanent One selects a member (constraint evaluation), promotes it (data loading), operates on it (computation), demotes it (result storage), and selects the next member. The constraint filter in CLA's fourth list — the conditions under which each event can fire — is the promotion mechanism. It takes a population and resolves it to the specific member that becomes temporary One.

This distinction clarifies several phenomena in computing that are otherwise described in domain-specific terms without their shared structure being visible.

Locking and concurrency control are mechanisms for serializing conflicting promotions to temporary One. When two threads attempt to write to the same file simultaneously, both are attempting to promote the same Infinity member to temporary One at the same time. A mutex ensures that only one promotion occurs at a time. The critical section is the duration of the temporary One promotion. Deadlock occurs when two permanent Ones (threads, which are themselves Infinity members acting as permanent Ones within their scope) each hold one resource as temporary One and wait for the other's resource to be released. The deadlock is a promotion conflict: each permanent One has promoted one resource and cannot proceed without promoting another that is already promoted by someone else.

Cache locality in CPU design is the hardware manifestation of temporary One promotion. When a population member is promoted — when its data is loaded into cache lines and registers — subsequent operations on nearby members are faster because the cache has anticipated the iteration pattern. Cache misses occur when the next temporary One promotion requires data that is not near the previous one. The CPU's memory hierarchy is optimized for the assumption that temporary One promotions follow predictable patterns — sequential iteration, strided access, temporal reuse. Random promotion patterns (pointer chasing, hash table lookup) defeat the cache because they violate the assumption that the next temporary One is near the previous one.

Context switching in operating systems is the cost of changing which permanent One is active. When the scheduler switches from one process to another, it is demoting one permanent One (the currently running process, which has had One-ness for the duration of its time slice) and promoting another. The cost — saving registers, flushing TLB entries, loading new page tables — is the cost of transferring the operational context of One-ness from one entity to another. The reason context switches are expensive is that One-ness is not a label; it is a physical state of the hardware, with data loaded, caches warm, and branch predictors trained. Transferring that state is transferring One-ness itself.

5. Zero Is the Boundary of What Can Become One

Zero-cardinality entities cannot be promoted to temporary One. They cannot be loaded, operated on, modified, or managed. They emit events into the system — they cause things to happen — but the system cannot cause things to happen to them. This is not a design restriction imposed by CLA. It is a statement about what lies outside the system's operational boundary.

The examples across domains make the nature of Zero concrete. In the operating system specification from [\[HOWL-COMP-12-2026\]](#), the BIOS is Zero because it executes before the OS exists. The OS cannot manage the BIOS because the OS does not yet exist when the BIOS acts. This is not a choice — it is a temporal impossibility. The Bootloader is Zero for the same reason. The OS begins at the moment when a Zero-cardinality group (Bootloader) emits an event (Control Transferred) that the first One-cardinality group (Kernel) receives.

In TCP, the Remote Host is Zero. The TCP stack cannot manage the remote host's behavior — cannot control when it sends data, how it responds to congestion signals, whether it respects window sizes, or whether it sends malformed segments. The network medium is Zero. TCP cannot prevent packet loss, duplication, reordering, or corruption. It can only detect these conditions after they have occurred and compensate. The application layer above TCP is Zero from TCP's perspective — TCP cannot control when the application writes data or how fast it reads.

In an automobile, the driver is Zero. The car cannot manage the driver — cannot prevent sudden acceleration requests, cannot control steering input, cannot ensure the driver is attentive. The road surface is Zero. The weather is Zero. The car's entire purpose is producing useful behavior in response to inputs from entities it fundamentally cannot control.

In a large language model performing token prediction, the training corpus is Zero — it was consumed during training and no longer exists at inference time, yet every weight in the model is a residue of its influence. Human intent is Zero — the model receives tokens that are a lossy projection of what the human actually wants, and it cannot observe or manage the intent behind those tokens. The rules of language are Zero — grammar, semantics, pragmatics exist as constraints on valid output, but the model has no language entity to consult, only statistical approximations baked into weights.

In every case, Zero groups share the same properties. They can only emit events, never receive them. Their events have the thinnest constraints in the specification — often just “true” or “previous event completed,” because there is no entity data to query. They are the source of the system's initial events — every system begins with an unconstrainable event from a Zero group, because the system's own operation cannot have caused its own starting condition. And they define the system's character more than any other cardinality, because coping with what cannot be controlled is what the system exists to do.

Zero is the boundary. Not a line drawn on a diagram, but an operational limit — the point beyond which promotion to One is impossible. Everything inside the boundary can become One (permanently or temporarily) and be operated on. Everything outside the boundary is Zero: referenced, depended upon, reacted to, but never managed.

6. The Fixed Interaction Patterns

When entities of different cardinalities interact, the interaction pattern is determined by the cardinalities involved. This determination is mechanical — it follows from the natures described in the preceding sections, not from design decisions.

One to Infinity is always fan-out. A permanent One operating on an Infinity population must iterate. It cannot address the population as a whole (Section 2 — One is the unit of work). It cannot address a specific member by identity alone (One relates to Infinity through constraint filtering — it specifies what properties the member must have, and the runtime resolves which member matches). The iteration — for each member matching the constraint, promote to temporary One, operate, demote — is fan-out. The OS init

system determining the runlevel and starting each service in that runlevel is fan-out. The TCP stack checking each segment for retransmission timeout is fan-out. The engine's ECU calculating injection timing for each cylinder is fan-out. The integral evaluating the antiderivative at each bound is fan-out. The pattern is identical because the cardinality interaction is identical.

Infinity to One is always convergence. When multiple Infinity members must contribute to a decision that a permanent One makes, each member is temporarily promoted to One, its relevant data is extracted, and the extracted data is aggregated into a single value that the permanent One operates on. The OS init system waiting for all filesystem mounts to complete before proceeding is convergence — each mount reports its status (temporary One during the report), the statuses are aggregated (all passed?), the aggregate (one boolean) is what init evaluates. The TCP receiver reassembling out-of-order segments is convergence — each segment contributes its sequence-number-ordered data, the reassembly produces one contiguous byte stream that the application reads. The definite integral evaluating $F(b) - F(a)$ is convergence — two evaluations (two temporary Ones) produce two values that are combined into one result.

Infinity to Infinity is always mutual constraint filtering. When two populations interact, neither can address the other's members directly. Both sides must be filtered by constraint to identify which members participate. A device becoming ready triggers the corresponding service to start — both Device and Service are populations, and the “corresponding” relationship is a constraint match (device type maps to service name). A process creating a pipe involves the Process population and the Pipe population — which process and which pipe are determined by constraint (the process that requested the pipe creation, the pipe allocated from the pool). The interaction is nested iteration: for each matching member in population A, for each matching member in population B, perform one operation on the pair.

Zero to anything is always one-way event injection. Zero groups emit events. They do not receive operations, responses, or modifications. The BIOS emits Power On Self Test completed. The remote host emits a SYN packet. The driver presses the brake pedal. The training corpus emitted examples during training. In every case, the event flows from Zero into the system, and the system processes it. No event flows back. There is no “response to the BIOS” or “modification of the training corpus.” The interaction is asymmetric by the nature of Zero.

One to One is always peer handoff. When two singletons interact, one completes a phase and enables the other to begin its phase. The Kernel's root switch enables the Init system to start. The Init system's DNS configuration enables the Display Server to start. These are sequential handoffs between peers. One singleton does not orchestrate another — if it did, the orchestrated singleton would be subordinate, raising the question of whether it should be an independent singleton or a subsystem of the orchestrator. Singletons are peers, and their interaction is linear sequencing or parallel independence.

Zero to Zero is pure sequencing with no entity data. When two Zero groups interact, neither has runtime data. The only possible constraint is whether the prior event completed. The BIOS transferring control to the Bootloader is one Zero event enabling

another Zero event. No entity data is read or written. The interaction is a fact about ordering, nothing more.

These six patterns are not a taxonomy that CLA proposes. They are consequences of the natures of the three cardinalities. One cannot address Infinity except by iteration (Section 2). Infinity cannot orchestrate One because it lacks population-level visibility (Section 3). Zero cannot receive events because it is outside the operational boundary (Section 5). The patterns follow from the natures. They are not design choices.

7. Unreliable Zero Creates Decision Points

A system's decision points — places where it must choose between alternatives rather than execute a deterministic sequence — do not arise randomly. They arise at specific structural locations, and those locations are predictable from the system's cardinality declarations.

The observation emerged from applying CLA across multiple domains. In the operating system, the nine behavior sets (scored decision points using utility AI) were almost all resource pressure responses: memory pressure, swap pressure, network congestion, log buffer pressure. In TCP, the five or more behavior sets — congestion control, retransmission timeout calculation, receive window management, delayed ACK timing, Nagle's algorithm — all exist because the network medium, the remote host, or the application layer behaves unpredictably. In Postfix, the policy decision behavior set exists because the remote sender is untrusted and DNS is unreliable. In the automobile, ABS and traction control exist because the road surface is unpredictable.

The pattern is: when a Zero-cardinality dependency is unreliable — when its events arrive at unpredictable times, with unpredictable content, or fail to arrive at all — the system must decide how to proceed without complete information. This decision cannot be deterministic, because deterministic action requires predictable input. The system must score alternatives: if the DNS lookup times out during SPF verification, accept the message (risk spam), reject it (risk false positive), or defer it (add latency). If free memory pages drop below a threshold, do nothing (risk OOM), compact (moderate intervention), reclaim (aggressive intervention), or invoke the OOM killer (extreme intervention). Each alternative has costs and benefits that depend on the current state, and the system must weigh them.

This is what CLA calls a behavior set: a scored evaluation of multiple possible actions using weighted considerations. The considerations are measurements of the system's current state — free page ratio, drop rate, idle time, error count. The weights and curves determine how urgently each measurement drives the decision. The winning behavior is the action with the highest composite score.

The claim is: the number and character of behavior sets in a system can be predicted from its unreliable Zero-cardinality dependencies. Each dimension of unreliability in an unmanaged dependency forces a decision point into existence. TCP has five Zero groups, several with multiple dimensions of unreliability (the network medium can lose packets,

reorder packets, duplicate packets, corrupt packets, and introduce variable delay — each dimension producing its own decision requirements), and TCP has at least five behavior sets. Postfix has two significantly unreliable Zero groups (Remote Sender and DNS) and has four to five behavior sets. The automobile has three unreliable Zero groups (Driver, Road, Weather) and has six to eight behavior sets.

Conversely, systems with reliable Zero dependencies have fewer behavior sets. A Navicat-style database GUI has two Zero groups (Database Server and User), but the Database Server is relatively reliable (it either responds or times out — the failure modes are simple), and the User's unreliability manifests not as unpredictable data but as unpredictable sequencing, which does not force scored decisions. Navicat has perhaps three behavior sets, the fewest of any system examined.

The large language model performing token prediction is the extreme case. Its Zero groups — Training Corpus, Human Intent, Language — are not unreliable in the way that network packets or road surfaces are unreliable. The training corpus is fixed. Human intent is unknown but stable for the duration of a prompt. Language rules are constant. The unreliability, such as it is, has been absorbed into the weights during training rather than requiring runtime decision-making. The result: one behavior set in the entire system (the sampling strategy), and at temperature zero, even that collapses to a deterministic argmax.

The relationship between Zero unreliability and behavior set count is not a coincidence or a correlation. It is structural. A deterministic system — one with no behavior sets, only forced actions — can exist only when every input is predictable enough that the correct response can be specified in advance. When an input becomes unpredictable, no single predetermined response is always correct, and the system must evaluate alternatives at runtime. The evaluation mechanism is the behavior set. The source of unpredictability is the Zero group. The behavior set exists because the Zero group is unreliable. Remove the unreliability, and the behavior set becomes unnecessary. Add a new dimension of unreliability, and a new behavior set (or new considerations in an existing one) becomes necessary.

This provides a practical prediction: when designing a system, enumerate its Zero-cardinality dependencies and assess each for reliability. Each unreliable dependency will require decision logic — a behavior set, a policy engine, a heuristic, a scoring function, whatever the domain calls it. The total count of such decision points is bounded by the number of unreliable Zero dependencies multiplied by their dimensions of unreliability. Everything else in the system can be deterministic sequencing.

8. The Cardinality Violation Principle

The natures described in Sections 2 through 5 are not guidelines. They are properties that assert themselves through consequences when violated. This section examines what happens when a system's actual behavior contradicts its cardinality declarations.

Infinity members acting as One. The most common and consequential violation occurs

when a member of an Infinity population acquires population-level visibility and begins making decisions that affect other members or the group as a whole. The defining test is: does this member's behavior set work identically with a population of one as with a population of one thousand? If yes, the member is operating as temporary One scoped to itself — legitimate, within the nature of Infinity. If no — if the member reads from the population, compares against other members, or makes decisions that route events to other members — the member is acting as One while declared as Infinity.

In distributed computing, this violation is the norm, and its consequences are the field's central problems. A peer-to-peer network consists of N nodes, each declared as a member of an Infinity population (all nodes are structurally equivalent). But each node makes routing decisions based on its view of other nodes — it reads from the population and makes decisions that affect other members. Every node is acting as a defacto One. The system has N competing singletons with no mechanism to resolve their competing orchestration decisions.

The result is the consensus problem. Consensus algorithms — Raft, Paxos, Practical Byzantine Fault Tolerance — are mechanisms for electing which Infinity member actually gets to be One for each decision round. The algorithm ensures that at most one member's view of the system state is authoritative at any given time. The rest defer. The cost of consensus — the message overhead, the latency, the leadership election protocol, the possibility of split-brain — is the cost of resolving a cardinality violation. The system declared all nodes as Infinity, but the work requires One, and the runtime must reconcile this.

Blockchain extends the pattern. Every mining or validating node is an Infinity member acting as One — reading the population of unconfirmed transactions, deciding which to include in a block, proposing the block as authoritative. Proof of work, proof of stake, delegated proof of stake — these are constraint mechanisms for resolving which Infinity member's temporary One-ness is accepted by the rest. The entire energy expenditure of proof-of-work mining is the cost of a cardinality violation: N entities competing for the right to be One, with the competition deliberately made expensive so that the resolution is unambiguous.

In concurrent programming on a single machine, the violation is smaller in scale but identical in structure. Two threads (Infinity members) attempting to write to the same shared variable (promoting the same resource to temporary One simultaneously) produce a race condition. The mutex is the resolution mechanism — it serializes promotions to temporary One, ensuring that only one thread's view of the resource is active at a time. The overhead of lock acquisition, the risk of deadlock, the complexity of lock-free algorithms — these are costs of resolving cardinality violations.

The principle generalizes: when an Infinity member violates its nature by assuming One-ness, the system must pay a cost to resolve the violation. The cost manifests differently in different domains — consensus overhead in distributed systems, lock contention in concurrent programming, election protocols in distributed databases, energy expenditure in blockchain — but it is always present and always proportional to the number of Infinity members simultaneously claiming One-ness.

A system can avoid these costs only by making the cardinality assignment explicit: designate one entity as One (a leader, a coordinator, a master), and let the Infinity population defer to it. This is why leader-based architectures (primary-replica databases, master-worker distributed systems, centralized schedulers) are simpler and faster than peer-based architectures. They align the runtime behavior with the cardinality natures. The leader is One. The replicas are Infinity. No violation occurs, so no resolution cost is incurred.

The tradeoff is that a designated One is a single point of failure — if the One fails, the system must promote an Infinity member to One (leader election), which temporarily re-introduces the violation until the election completes. High availability in distributed systems is the engineering of rapid, bounded-cost cardinality violation resolution when the permanent One fails.

9. Zero-to-One Transitions as Architectural Phase Changes

A system's Zero-cardinality groups define what it cannot manage. When one of these groups transitions to One — when the system begins managing what it previously only reacted to — the system undergoes its most fundamental architectural transformation.

The transition from Zero to One is not an incremental feature addition. It redefines the system's boundary, its identity, and its relationship to its environment. CLA's closedness property — the ability to add new components by adding entries to the four lists without modifying existing entries — does not apply here. A cardinality transition modifies the EntityGroup declaration itself. Every flow, every constraint, every event that referenced the formerly-Zero group as an external event source must be reconsidered, because the group's events are now internally generated and manageable rather than externally received and uncontrollable.

The clearest contemporary example is autonomous driving. In a conventional automobile's CLA specification, the Driver is Zero. The car receives steering input, throttle requests, and braking commands as events from an entity it cannot manage. The car's entire interaction with the Driver is reactive — events flow from Driver into the car's singletons, never from the car back to the Driver in a way that modifies Driver behavior. Advanced driver assistance systems (ADAS) — lane departure warnings, collision alerts, adaptive cruise control — are attempts to add events that flow toward the Zero-cardinality Driver, but since Driver is Zero, these events can only be informational. The car can beep. It cannot steer the driver's hands.

When the Driver transitions from Zero to One, the car now manages the driving function. Steering, throttle, and braking are no longer external events — they are actions generated internally by a singleton orchestrator. The perception system (previously receiving Zero-cardinality road and weather events passively) now feeds into a planning system (One) that generates driving commands. The entire flow structure of the vehicle's specification changes. What was one-way event reception from an unmanaged source becomes a closed loop: perceive, plan, act, perceive. The system's boundary has moved. What was outside is now inside.

This pattern appears at every major architectural inflection point in computing history. The transition from client-server to cloud computing is the Server transitioning from Zero (the application connects to it but doesn't manage it) to One (the application provisions, scales, and manages its own infrastructure). The transition from static web pages to web applications is Content transitioning from Zero (the server serves fixed files) to One (the application generates content dynamically, managing it as a runtime entity). The transition from compiled languages to language workbenches is the Grammar transitioning from Zero (the compiler implements a fixed language definition) to One (the workbench manages the grammar as a mutable runtime entity). The transition from traditional databases to embedded databases is the Database Engine transitioning from Zero (the application connects to an external server) to One (the application contains and manages the storage engine directly).

In each case, the transition redefines what the system is. A web server that manages its own content is a fundamentally different system from one that serves static files, even if the HTTP protocol and network handling are identical. The Zero-to-One transition on one EntityGroup changes the system's character more than adding a hundred new Infinity populations would, because it changes what the system controls rather than what it contains.

The reverse transition — One to Zero — is equally significant and often deliberate. Microservice architecture is the systematic transition of subsystems from One (monolith components managed internally) to Zero (external services that the application references but does not manage). Each extraction moves a capability outside the system's boundary. The application can no longer manage the extracted service's lifecycle, deployment, or internal state. It can only send requests and receive responses — event injection from a Zero-cardinality dependency. The costs and benefits of microservices are the costs and benefits of making previously-One groups into Zero groups: gained independence and isolation, lost direct management and guaranteed availability.

10. Cross-Domain Universality

The properties described in the preceding sections were identified through analysis of software systems — operating systems, network protocols, automobiles, language models, database tools, mail transfer agents. This section demonstrates that the same properties appear in systems that process information outside of software, because the properties are intrinsic to information processing rather than to any particular implementation substrate.

A hospital. Patient is Infinity — a population of individuals, each temporarily promoted to One during a consultation, procedure, or diagnostic. Doctor is also Infinity, but each doctor acts as temporary One with population-level visibility scoped to their patient list — a defacto permanent One within their ward or practice. Hospital Administration is permanent One — the singleton orchestrating the populations. Disease is Zero — the hospital references diseases, diagnoses them, treats their effects, but cannot manage disease itself. Disease exists outside the hospital's operational boundary. The hospital cannot prevent a novel pathogen from appearing, cannot control mutation rates, cannot manage the epi-

demiological dynamics that determine which diseases its patients present with. It can only react.

The behavior sets are where the hospital makes scored decisions, and they concentrate exactly where the theory predicts: at unreliable Zero-cardinality dependencies. Triage is a behavior set forced into existence by the unreliability of Patient Arrival (a Zero event — the hospital cannot control when patients arrive or how sick they are). Emergency department staffing is a behavior set responding to unpredictable demand. Antibiotic selection is a behavior set responding to uncertain pathogen identification (the pathogen's resistance profile is a Zero-cardinality fact the hospital must determine, not manage).

Resource pressure response appears here too. Bed management under capacity pressure is structurally identical to the OS memory manager under page pressure — score whether to do nothing, discharge early, divert ambulances, or open overflow capacity, based on occupancy ratio, incoming patient rate, and acuity distribution. The considerations, curves, and weights would differ from the memory manager's, but the structure — a singleton scoring alternatives in response to a ratio approaching a limit — is the same.

A classroom. Teacher is permanent One — the singleton orchestrator. Students are Infinity — a population of learners, each temporarily promoted to One when called upon, assessed, or given individual feedback. Curriculum is Zero — the teacher references it, follows it, adapts to it, but does not manage it. The school board, the state standards committee, the textbook publisher — these determine the curriculum, and the teacher's relationship to it is reactive. Lesson planning is the teacher's mechanism for translating Zero-cardinality curriculum events into a sequence of One-to-Infinity fan-outs (teaching the class) and Infinity-to-One convergences (assessing whether the class understood).

The behavior set prediction holds. The teacher's decision points concentrate at unreliable Zero dependencies. A student struggling unexpectedly (Student behavior is a Zero-cardinality event from the teacher's orchestration perspective — the teacher cannot directly control whether a student understands) forces a scored decision: re-explain, use a different analogy, assign peer tutoring, move on. Classroom management is a behavior set responding to unpredictable student behavior. Pacing decisions — speed up, slow down, skip ahead, revisit — respond to the unreliable signal of student comprehension.

A supply chain. Warehouse is One (or Infinity if multiple warehouses, with a Logistics Coordinator as One above them). Shipment is Infinity. Supplier is Zero — the supply chain depends on suppliers but does not manage their internal operations, production schedules, or quality control processes. Customer Demand is Zero — unpredictable, uncontrollable, only observable after the fact.

Inventory management is the behavior set, and it is a pressure response: score whether to reorder, expedite, discount, or write off based on stock ratio, demand velocity, lead time, and carrying cost. The structure is the OS memory manager again — a singleton scoring alternatives based on a resource ratio approaching a limit, forced into existence by an unreliable Zero dependency (Customer Demand fluctuating unpredictably).

The bullwhip effect in supply chains — where small demand fluctuations at the consumer end amplify into large inventory swings upstream — is a cardinality violation conse-

quence. Each node in the supply chain (each warehouse, each distributor) acts as a defacto One, making independent inventory decisions based on its local view. But the decisions interact: one node's order is another node's demand signal. N nodes each acting as One, making population-affecting decisions without population-level coordination, produces oscillation. The resolution mechanisms — vendor-managed inventory, demand signal sharing, collaborative planning — are supply chain analogues of consensus protocols: mechanisms for ensuring that inventory decisions are coordinated rather than independently asserted by N competing defacto-Ones.

Calculus. The integrand function is One — the single thing being operated on. The infinitesimal slices (dx) are Infinity — a population of intervals, each temporarily promoted to One during evaluation (compute $f(x)$ at this point, multiply by dx , accumulate). The theorem justifying the operation — the Fundamental Theorem of Calculus, or the power rule, or integration by parts — is Zero. The mathematician references the theorem, depends on it for the validity of the operation, but does not manage it. The theorem exists outside the computation's operational boundary.

The definite integral's structure is fan-out followed by convergence. One function fans out to Infinity slices. Each slice is evaluated (temporary One). The evaluations converge to one sum. The Fundamental Theorem reveals that this fan-out/convergence cycle is closed: differentiation fans out (one function becomes information about its rate of change at every point — One to Infinity), and integration converges (the rates of change at every point are accumulated back to one function — Infinity to One). The theorem is the statement that complete fan-out followed by complete convergence returns to the starting point. This is the same structural guarantee as TCP's reliability: all segments fanned out into the network must converge at the receiver, or the operation is incomplete.

The universality claim. These examples span healthcare, education, logistics, and mathematics. In each case, the same properties appear: One as the unit of work, Infinity as repetition of One requiring reduction, Zero as the boundary of what can become One, unreliable Zero dependencies creating decision points, and cardinality violations producing the domain's hardest coordination problems. The properties appear because they are intrinsic to the processing of information, regardless of whether the processor is a CPU, a surgeon, a teacher, a warehouse manager, or a mathematician.

No domain-specific training is required to identify these properties. The procedure is mechanical: enumerate what the system references but cannot manage (Zero), what exists as exactly one instance coordinating the rest (One), and what exists as a population of instances managed by the singletons (Infinity). The interaction patterns, decision point locations, and violation consequences follow from the natures of the cardinalities, not from domain expertise.

11. Implications

The three cardinalities and their natures are not a framework to be adopted or a method to be learned. They are properties to be recognized. They exist in every information-

processing system whether the system's designers are aware of them or not. Naming them provides specific, practical benefits.

A system that names its Zero groups explicitly gains visibility into its boundaries and security surface. Every security mechanism in every system is a constraint on events from Zero-cardinality groups. Authentication constrains who the Zero-cardinality user claims to be. Authorization constrains what the Zero-cardinality user requests. Input validation constrains the shape of Zero-cardinality input. Rate limiting constrains the frequency of Zero-cardinality events. Encryption protects data traversing a Zero-cardinality network medium. Enumerating the Zero groups enumerates the attack surface. An event from a Zero group with insufficient constraints is an unprotected boundary. This enumeration is mechanical — scan the entity group list for Zero cardinality, then scan the constraint list for events from those groups. Every gap is a potential vulnerability.

A system that distinguishes permanent One from temporary One gains clarity about its concurrency architecture. Every permanent One is a coordination point. Every temporary One promotion is a potential contention point. The number of permanent Ones determines the system's concurrency topology — one permanent One is a centralized system, N permanent Ones is a distributed system requiring consensus, and the costs of each are predictable from the cardinality structure. Recognizing when an Infinity member has become a defacto permanent One — when it has acquired population-level visibility and makes decisions affecting other members — predicts where the system's coordination costs will concentrate.

A system that counts its behavior sets against its unreliable Zero dependencies gains confidence that it has found all of its decision points. If the count of behavior sets is less than the count of unreliable Zero dependency dimensions, a decision point is missing — the system is handling unpredictable input deterministically, which means it will be wrong when the input deviates from the assumed case. If the count of behavior sets is greater, the system may be making decisions that could be deterministic, adding unnecessary complexity.

A system that identifies its cardinality violations gains predictive power about where its hardest problems will concentrate. The consensus problem in distributed systems, the deadlock problem in concurrent programming, the bullwhip effect in supply chains, the split-brain problem in distributed databases — these are all consequences of Infinity members acting as One without coordination mechanisms. Identifying the violation predicts the problem. Designing the resolution mechanism — leader election, locking, demand signal sharing, quorum protocols — becomes a targeted response to a named structural condition rather than an ad hoc reaction to observed failures.

These benefits do not require adopting CLA or any other specification method. They require only the recognition that every information-processing system has Zero-cardinality dependencies it cannot manage, One-cardinality operations it performs one at a time, and Infinity-cardinality populations it must reduce to One before operating. The natures of these cardinalities — what they can and cannot do, how they interact, what happens when their natures are violated — are fixed. They are properties of information processing itself. Making them visible is the first step. What follows from visibility — better architecture,

fewer surprises, targeted engineering effort — follows naturally, because the properties were always there, shaping the system’s behavior whether they were named or not.

References

[[HOWL-COMP-12-2026](#)] “Closed Loop Architecture: A Complete OS in Four Flat Lists.” HOWL-COMP-12-2026. June 2026. DOI: 10.5281/zenodo.20615398.

[[HOWL-COMP-11-2026](#)] “Name Driven Development.” HOWL-COMP-11-2026. June 2026.

[[HOWL-COMP-1-2026](#)] “The Execution Pipeline.” HOWL-COMP-1-2026. January 2026.

HOWL-INFO-11-2026. The Relationship of Zero, One, and Infinity in Information Processing: The Intrinsic Cardinalities of Computation.

[[HOWL-INFO-11-2026](#)] The Relationship of Zero, One, and Infinity in Information Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-11-2026>

[[HOWL-INFO-1-2026](#)] Multi-Dimensional Information Indexing. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO-1-2026>

[[HOWL-INFO-2-2026](#)] The Scales Method. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-2-2026>

[[HOWL-INFO-3-2026](#)] The Pseudo-Socratic Method. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO-3-2026>

[[HOWL-INFO-4-2026](#)] Howland’s Axiom of Information Locality. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-4-2026>

[[HOWL-INFO-5-2026](#)] Coherence as Information Phenomenon. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO-5-2026>

[[HOWL-INFO-6-2026](#)] How to Write Technical Documents. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO-6-2026>

[[HOWL-INFO-7-2026](#)] How to Review Technical Writing. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO-7-2026>

[[HOWL-INFO-8-2026](#)] LLMs are not Tools, LLMs are Maybe-Tools. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-8-2026>

[[HOWL-INFO-10-2026](#)] Closing Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-10-2026>

[[HOWL-COMP-12-2026](#)] Closed Loop Architecture. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP-12-2026>

[[HOWL-COMP-11-2026](#)] Name Driven Development. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP-11-2026>

[[HOWL-COMP-1-2026](#)] Implementing a Tall-Infra Data-Only Execution System .
Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-1-2026>